

Introducción a la IA Aplicando Agentes

Percepción, decisión y acción con modelos de lenguaje

Objetivos de aprendizaje

Aprendizaje Esperado · Fundamentos de agentes de IA aplicados con modelos de lenguaje

- **Comprender** qué es un agente de IA y en qué se diferencia de un LLM o chatbot simple.
- **Identificar** los componentes de un agente: modelo, herramientas, memoria y orquestación.
- **Aplicar** el patrón ReAct (razonar y actuar) y el uso de herramientas (function calling).
- **Reconocer** tipos de agentes, patrones de diseño, casos de uso y riesgos al construir agentes.

Agenda

Cuatro módulos: de la definición de agente al uso responsable

1

¿Qué es un Agente de IA?

Definición, LLM vs agente y componentes · 15 min

2

Razonamiento y Acción

Function calling, patrón ReAct y frameworks · 15 min

3

Aplicación y Riesgos

Casos de uso, buenas prácticas y riesgos · 15 min

4

Tipos y Patrones

Taxonomía de agentes y patrones de diseño avanzados · 10 min

¿Qué es un Agente de IA?

De responder preguntas a percibir, decidir y actuar

LLM vs Agente

Componentes

Ejemplo

Chatbot vs Agente de IA

Mismo modelo de lenguaje, un ciclo de trabajo distinto

LLM / CHATBOT SIMPLE

Responde con base en el texto de entrada

- Recibe una pregunta y genera una respuesta de texto.
- No ejecuta acciones ni consulta el mundo exterior.
- La conversación termina en la respuesta generada.

AGENTE DE IA

Percibe, decide y actúa para lograr un objetivo

- Analiza la tarea y decide qué pasos seguir.
- Puede usar herramientas: buscar, calcular, consultar una API.
- Repite el ciclo hasta completar el objetivo.

PERCIBE → DECIDE → ACTÚA

Componentes de un agente

Cuatro piezas conectadas entre sí dentro de un mismo ciclo de trabajo

1

Modelo de lenguaje

Razona sobre la tarea y decide los siguientes pasos.

2

Herramientas (tools)

Funciones externas que el agente puede invocar.

3

Memoria

Mantiene contexto entre turnos de la conversación.

4

Orquestación

Coordina el ciclo de razonamiento, acción y observación.

Cómo Razona y Actúa un Agente

De la decisión a la ejecución de herramientas

Function calling

Patrón ReAct

Frameworks

Function calling y uso de herramientas

Un flujo de cuatro pasos que se repite hasta obtener la respuesta final

1

Definición

El modelo recibe una lista de funciones disponibles, con su nombre y parámetros.

2

Decisión

El modelo decide si necesita llamar a una función y con qué argumentos.

3

Ejecución

El programa ejecuta la función real y obtiene un resultado.

4

Respuesta

El resultado se entrega de vuelta al modelo para continuar el razonamiento.

Regla clave: el modelo nunca ejecuta código — solo decide qué función llamar; el programa la ejecuta.

Patrón ReAct: razonar y actuar

Un ciclo iterativo hasta completar el objetivo

```
Pensamiento: "Necesito el clima de Santiago"  
Acción: buscar_clima("Santiago")  
Observación: "18°C, despejado"
```

```
Pensamiento: "Ya tengo la información,  
puedo responder"  
Respuesta final: "En Santiago hay 18°C  
y está despejado."
```

El agente repite este ciclo — pensar, actuar, observar — tantas veces como sea necesario antes de responder.

ReAct (Reason + Act) fue propuesto por Yao et al. (2022) y es la base de la mayoría de los frameworks de agentes actuales.

Frameworks para construir agentes

Un panorama de las librerías más usadas

LangChain

Librería más popular y flexible para construir agentes con herramientas.

LangGraph

Extiende LangChain para flujos de agentes más complejos, como grafos de estados.

OpenAI Agents SDK

SDK oficial de OpenAI para construir agentes con sus modelos.

CrewAI

Orientado a equipos de agentes que colaboran en una tarea.

Ejemplo 1: agente con una herramienta

El caso más simple: un modelo, una herramienta y el ciclo ReAct

```
# Ejemplo 1: agente básico con una herramienta
from langchain.agents import initialize_agent, Tool
from langchain_groq import ChatGroq

llm = ChatGroq(model="openai/gpt-oss-20b")

def calculadora(expresion):
    return eval(expresion)

tools = [Tool(name="Calculadora",
              func=calculadora,
              description="Resuelve operaciones matemáticas")]

agente = initialize_agent(tools, llm,
                          agent="zero-shot-react-description")
agente.run("¿Cuánto es 24 * 7?")
```

funciones para crear el agente y sus herramientas
cliente del modelo de lenguaje (Groq, gratuito)

se crea el modelo que va a razonar

función normal de Python: la herramienta
evalúa el texto como una operación matemática

nombre que el modelo usará para invocarla
función de Python que se ejecuta al llamarla
le dice al modelo cuándo usarla

arma el agente con el modelo y las herramientas
este modo aplica el patrón ReAct
el agente razona, decide usar la tool y actúa

Conceptos aplicados: modelo de lenguaje (LLM) · herramienta (tool) · patrón ReAct (zero-shot-react-description)

Ejemplo 2: dos herramientas y memoria

Parte 1/2 · Se definen las herramientas y la memoria de conversación

```
# Ejemplo 2 (1/2): herramientas y memoria de conversación
from langchain.agents import initialize_agent, Tool, AgentType # agente, envoltorio de tool y tipos
from langchain.memory import ConversationBufferMemory          # guarda el historial de la conversación
from langchain_groq import ChatGroq                          # cliente del modelo (Groq, gratuito)

llm = lChatGroq(model="openai/gpt-oss-20b")                  # modelo que va a razonar y decidir
memoria = ConversationBufferMemory(memory_key="chat_history") # aquí se acumula el historial del chat

def buscar_libro(t):                                         # tool 1: recibe el título de un libro como texto
    return "Disponible" if t == "Sapiens" else "En préstamo" # respuesta simulada del catálogo

def renovar_prestamo(c):                                     # tool 2: recibe el código de un préstamo activo
    return f"Préstamo {c} renovado por 7 días más"          # respuesta simulada de la renovación
```

Conceptos aplicados: memoria de conversación (ConversationBufferMemory) · dos funciones de Python como herramientas

Ejemplo 2: dos herramientas y memoria

Parte 2/2 · Tarea cotidiana del alumno: revisar y renovar un préstamo

```
# Ejemplo 2 (2/2): armar el agente con memoria y ejecutarlo
tools = [
    Tool(name="BuscarLibro", func=buscar_libro,
          description="Busca disponibilidad de un libro"),
    Tool(name="RenovarPrestamo", func=renovar_prestamo,
          description="Renueva un préstamo activo"),
]

# se agrupan las dos herramientas del agente
# el modelo la usa si preguntan por un libro
# el modelo la usa si piden renovar

agente = initialize_agent(
    tools, llm,
    memory=memoria,
    agent=AgentType.CONVERSATIONAL_REACT_DESCRIPTION,
)
# herramientas y modelo definidos antes
# conecta la memoria al agente
# modo conversacional: usa el historial

agente.run("¿Está disponible Sapiens? Si no, renueva mi préstamo B-045") # una consulta, dos pasos
```

Conceptos aplicados: varias herramientas en un mismo agente · AgentType.CONVERSATIONAL_REACT_DESCRIPTION · memoria activa

Ejemplo 3: varias herramientas y prompt guiado

Parte 1/2 · Herramientas declaradas con el decorador @tool

```
# Ejemplo 3 (1/2): herramientas con @tool y docstring
from langchain_core.tools import tool                                # decorador que convierte una función en tool

@tool
def buscar_libro(t: str) -> str:                                    # recibe el título del libro como texto
    """Consulta si un libro está disponible en catálogo."""      # el modelo lee esto para saber cuándo usarla
    return "Disponible" if t == "Sapiens" else "En préstamo"      # respuesta simulada del catálogo

@tool
def sala_disponible(hora: str) -> str:                             # recibe la hora deseada como texto
    """Revisa si hay una sala de estudio libre a esa hora."""
    return "Sala 204 libre" if hora != "18:00" else "Sin salas"   # respuesta simulada

@tool
def reservar_sala(sala: str, hora: str) -> str:                  # recibe la sala y la hora a reservar
    """Reserva una sala de estudio para el alumno."""
    return f"{sala} reservada a las {hora}"                       # confirmación simulada de la reserva
```

Conceptos aplicados: herramientas declarativas (@tool) · tipado de parámetros · docstring como descripción para el modelo

Ejemplo 3: varias herramientas y prompt guiado

Parte 2/2 · Tarea cotidiana del alumno: buscar un libro y reservar una sala

```
# Ejemplo 3 (2/2): prompt de sistema, agente y ejecución
from langchain.agents import AgentExecutor, create_tool_calling_agent # arma y ejecuta el agente
from langchain_core.prompts import ChatPromptTemplate # define las instrucciones del agente
from langchain_groq import ChatGroq # cliente del modelo (Groq, gratuito)

tools = [buscar_libro, sala_disponible, reservar_sala] # las 3 herramientas del Ejemplo 3 (1/2)

prompt = ChatPromptTemplate.from_messages([
    ("system", "Ayudas con libros y salas. Confirma antes de reservar."), # reglas fijas del agente
    ("human", "{input}"), # aquí se inserta la pregunta del alumno
    ("placeholder", "{agent_scratchpad}"), # aquí anota sus pasos intermedios
])

llm = ChatGroq(model="llama-3.1-70b-versatile") # modelo más grande: razona mejor con 3 tools
agente = create_tool_calling_agent(llm, tools, prompt) # une modelo, herramientas y prompt
ejecutor = AgentExecutor(agent=agente, tools=tools, verbose=True) # ejecuta el ciclo y muestra los pasos

ejecutor.invoke({"input": "Necesito Sapiens y una sala a las 16:00"}) # encadena las 3 herramientas
```

Conceptos aplicados: prompt de sistema (ChatPromptTemplate) · AgentExecutor · varias herramientas encadenadas en una consulta

Aplicación, Buenas Prácticas y Riesgos

De la teoría a un uso responsable

Casos de uso

Buenas prácticas

Riesgos

Casos de uso aplicados

Dónde se están usando agentes de IA hoy

1

Asistentes de atención

Responden consultas de clientes y escalan casos complejos a personas.

2

Automatización de tareas

Ejecutan flujos repetitivos: agendar, completar formularios, generar reportes.

3

Agentes de búsqueda documental

Consultan documentos o bases de conocimiento para responder con contexto.

Buenas prácticas, límites y riesgos

Ningún agente debería operar sin supervisión humana

RIESGOS A CONSIDERAR

- Alucinaciones: el modelo puede inventar información o resultados.
- Costos de uso: cada llamada al modelo y a las herramientas tiene un costo asociado.
- Seguridad: una herramienta mal restringida puede ejecutar acciones no deseadas.

BUENAS PRÁCTICAS

- Definir con claridad qué herramientas puede usar el agente y con qué permisos.
- Supervisar y revisar las acciones del agente, especialmente en tareas críticas.
- Establecer límites de costo y de intentos, para evitar ciclos infinitos.

Tipos de Agentes y Patrones de Diseño

De la taxonomía clásica a los patrones que coordinan sistemas complejos

Tipos de agentes

Reflexión y planificación

Multiagente

Tipos de agentes de IA

Cinco categorías según su inteligencia y adaptabilidad

1

Reflejo simple

Actúa solo según la entrada actual, sin memoria del pasado. Ejemplo: un termostato.

2

Basado en modelo

Mantiene una representación interna del entorno para decidir mejor. Ejemplo: aspiradora robot que mapea la casa.

3

Orientado a objetivos

Evalúa sus acciones según qué tan cerca lo dejan de una meta. Ejemplo: agente que planifica una ruta de viaje.

4

De aprendizaje

Mejora su desempeño con el tiempo a partir de datos y retroalimentación. Ejemplo: sistema de recomendación.

5

Sistemas multiagente

Varios agentes colaboran o compiten para resolver una tarea. Ejemplo: agentes que negocian precios entre sí.

Patrones de diseño: reflexión y planificación

Dos formas de mejorar un resultado iterando antes de responder

REFLEXIÓN

1

Generar

El modelo produce una primera versión de la respuesta.

2

Criticar

El mismo modelo revisa el resultado y detecta errores o vacíos.

3

Reescribir

Se corrige la versión anterior aplicando la crítica recibida.

PLANIFICACIÓN

1

Crear plan

El modelo lista los pasos necesarios.

2

Ejecutar paso

Se llama a la herramienta del paso actual.

3

Observar

Se analiza el resultado obtenido.

4

Repetir o responder

Se continúa o se entrega la respuesta final.

Diferencia clave: la reflexión mejora un resultado ya generado; la planificación decide qué pasos ejecutar antes de actuar.

Patrones de colaboración multiagente

Cómo se organizan varios agentes especializados para una tarea

1

Secuencial

Investigador → Diseñador → Escritor. Cada agente entrega su resultado al siguiente. Fácil de depurar y predecible.

2

Paralelo

Investigador ⇒ Diseñador (a la vez) → Escritor combina resultados. Reduce tiempo, suma complejidad de coordinación.

3

Jerárquico

Gerente → (Investigador, Diseñador, Escritor). Un agente coordinador reparte y revisa el trabajo de los demás.

¿Es un agente de IA?

Para cada situación, determina si corresponde a un LLM simple o a un agente

#	Situación	¿LLM simple o agente?
1	Un chatbot responde preguntas frecuentes usando solo el texto que recibe.	LLM simple
2	Un sistema consulta el clima en una API y decide qué ruta recomendar.	Agente
3	Un asistente redacta un correo a partir de instrucciones del usuario.	LLM simple
4	Un sistema busca en documentos, resume resultados y agenda una reunión.	Agente
5	Un modelo responde con el resumen de un texto que se le entrega.	LLM simple

Mecánica: pregúntate si el sistema solo genera texto, o si además decide y ejecuta acciones sobre el mundo.

Tabla resumen de la sesión

Todo lo aprendido hoy, en una sola mirada

Concepto	Descripción	Ejemplo
Agente de IA	Percibe, decide y actúa para lograr un objetivo	Agente que busca y reserva un vuelo
Tool / function calling	Función externa que el modelo puede invocar	Calculadora, buscador, API
Patrón ReAct	Ciclo de pensar, actuar y observar	Pensamiento → Acción → Observación
Memoria	Contexto que se mantiene entre turnos	Recordar el nombre del usuario
Framework	Librería para construir agentes	LangChain, LangGraph, CrewAI

Lo que se llevan de hoy

Cuatro ideas para anclar el aprendizaje

1

Percibir, decidir, actuar

Un agente no solo responde: decide qué hacer y ejecuta acciones mediante herramientas.

2

El ciclo ReAct

Pensar, actuar y observar, repitiendo hasta lograr el objetivo.

3

Supervisión humana

Alucinaciones, costos y seguridad exigen revisar y limitar lo que un agente puede hacer.

4

Próxima parada: talleres

¿Qué viene después?

El recorrido completo del curso de IA con Agentes

1

Sesión Teórica

Qué es un agente y cómo razona

2

Taller Práctico 1

Agente básico con una herramienta

3

Taller Práctico 2

Múltiples herramientas y memoria

4

Tarea evaluada

Corrección a 80 alumnos

Próxima actividad: Taller Práctico 1 — construir tu primer agente.

¿Preguntas?

BIBLIOGRAFÍA OBLIGATORIA

- Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models.
- LangChain Documentation — python.langchain.com

BIBLIOGRAFÍA COMPLEMENTARIA

- Groq Documentation — console.groq.com/docs
- OpenAI — Function calling guide, platform.openai.com/docs
- Wyss, M. (2025). Agentes de IA: Curso completo — Medium.

PRÓXIMA SESIÓN

Taller Práctico 1: agente básico con herramientas

¡Gracias!

Completa nuestra encuesta para mejorar futuros talleres



Escanea el código QR con tu celular